VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

From Linear Models to Modern Sequence Models

A comparative study for Fashion-MNIST image classification

CO3133 Deep Learning and Its Applications

Group NNs | Semester 261 | Instructor: Lê Thành Sách

2352778	**Nguyễn Nhật Nam**
2352824	**Hồ Hồng Phúc Nguyên**
2353053	**Phan Quốc Đại Sơn**

HO CHI MINH CITY · OCTOBER 2026

CO3133 / ASSIGNMENT 1

Presentation roadmap

01

Problem & dataset

02

Methods & architectures

03

Experimental setup

04

Results & comparison

05

Error analysis

06

Conclusion & future work

01 / PROBLEM AND DATA

Research problem & motivation

Can architecture and input representation change how effectively a network classifies the same 28×28 image?

- Compare predictive quality: accuracy and macro-F1
- Compare costs: parameters, training time, batched forward time
- Understand how spatial and sequence inductive biases influence errors

Task: single-label classification into 10 fashion categories; models produce 10 logits.

01 / PROBLEM AND DATA

Fashion-MNIST: dataset overview

70,000
Total grayscale images

28 × 28
Pixels, 1 channel

10
Balanced categories

Stratified train / validation / test split



54,000 train 6,000 validation 10,000 official test

Seed 42 · 5,400 train + 600 validation + 1,000 test images per class.

01 / PROBLEM AND DATA

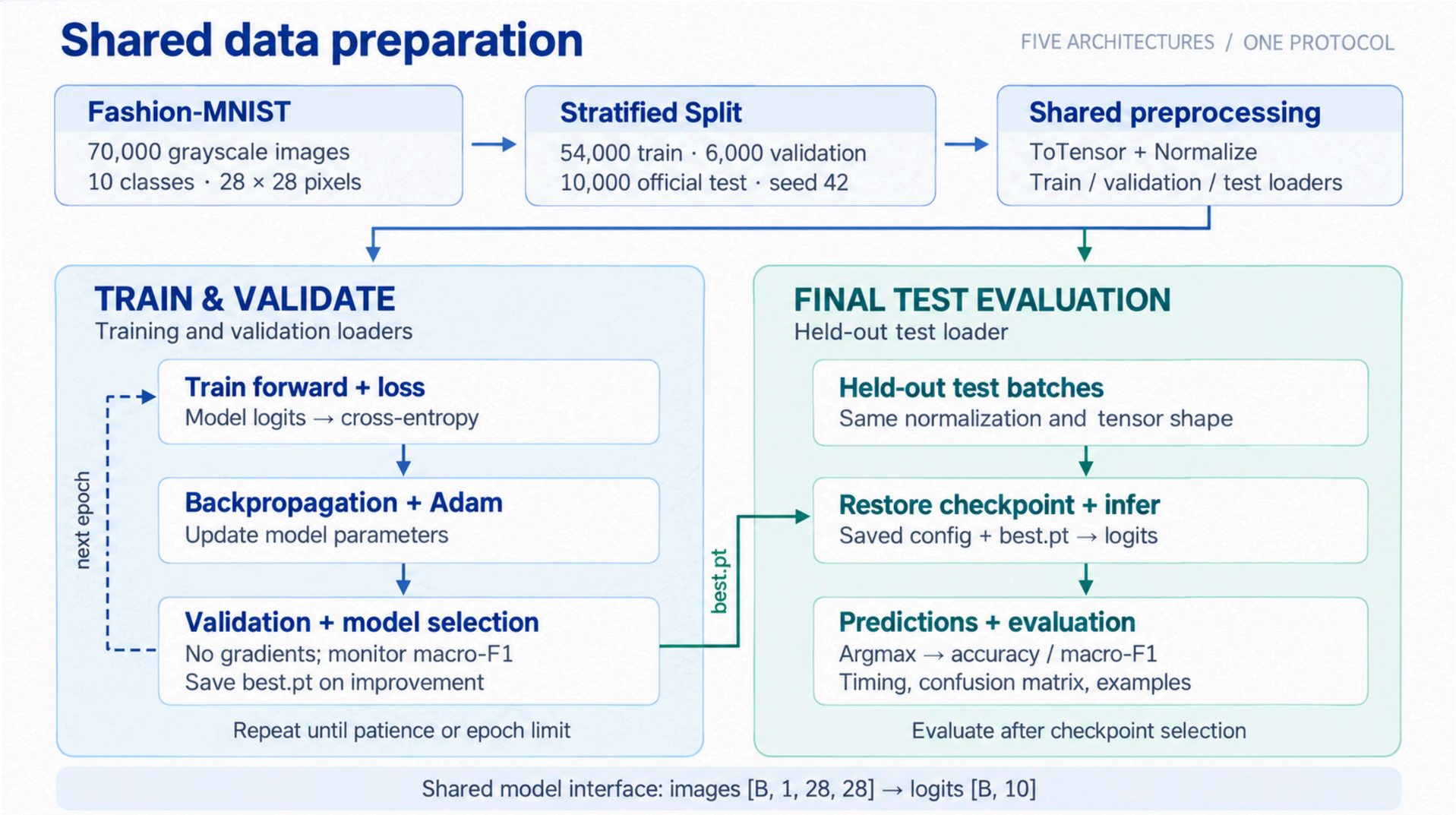
Data preparation & leakage control

- Fixed stratified split (seed 42); identical indices across all models
- ToTensor: uint8 pixels → floating-point tensor
- Normalize using training-only statistics: $\mu = 0.286209$, $\sigma = 0.353160$
- No data augmentation, resizing or padding
- Best checkpoint chosen on validation macro-F1; test set held out

Image tensor shape: [B, 1, 28, 28]. The report verifies index-level separation.

02 / METHODOLOGY

Shared model training pipeline



02 / METHODOLOGY

Five model families, seven configurations

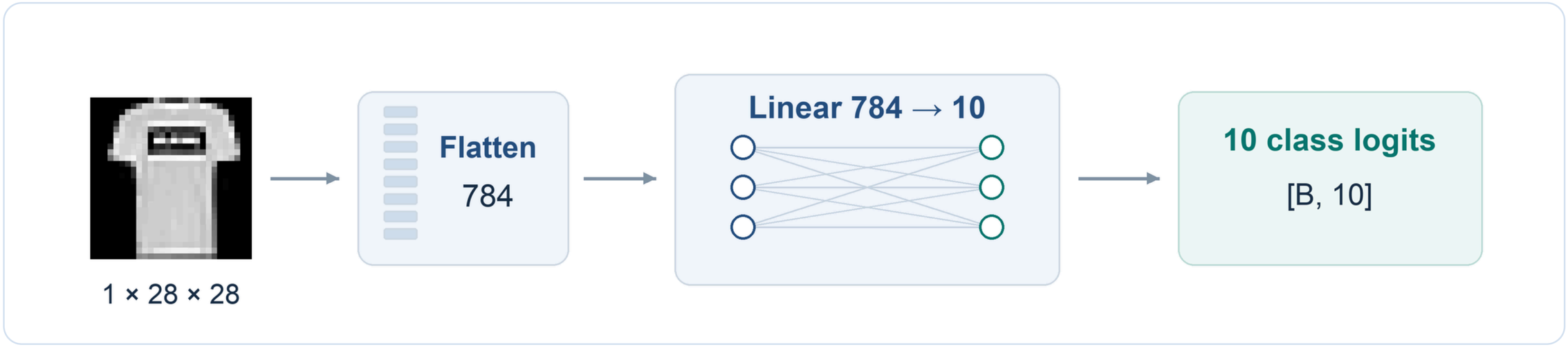
Linear	Flatten → affine mapping	**7,850**
MLP	Dense nonlinear layers	**235,146**
CNN	Local convolutions + pooling	**50,378**
LSTM	Rows / columns / patches	**82,186–92,938**
Transformer	Patch tokens + self-attention	**540,170**

All models trained from scratch; the three LSTM variants use different image-to-sequence representations.

02 / METHODOLOGY — ARCHITECTURE

Linear classifier — architecture

Flattened image and a single affine prediction layer



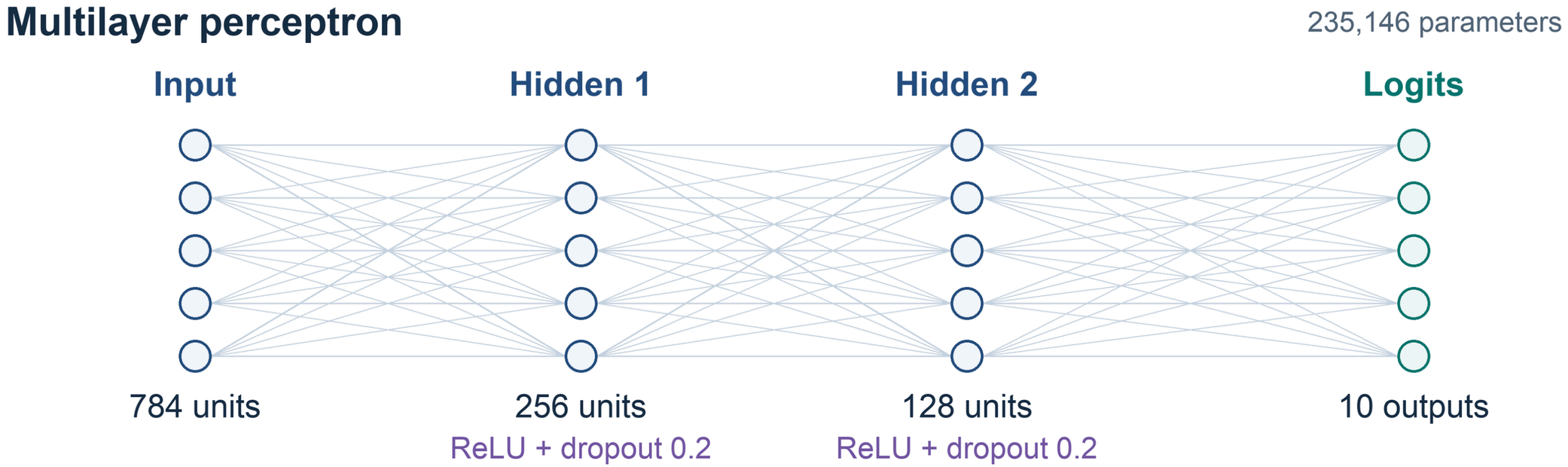
28×28 grayscale	Flatten → 784	Linear 784 → 10	10 class logits

7,850 parameters · No hidden layer · Used as the linear baseline.

02 / METHODOLOGY — ARCHITECTURE

Multilayer perceptron — architecture

Dense nonlinear feature transformation



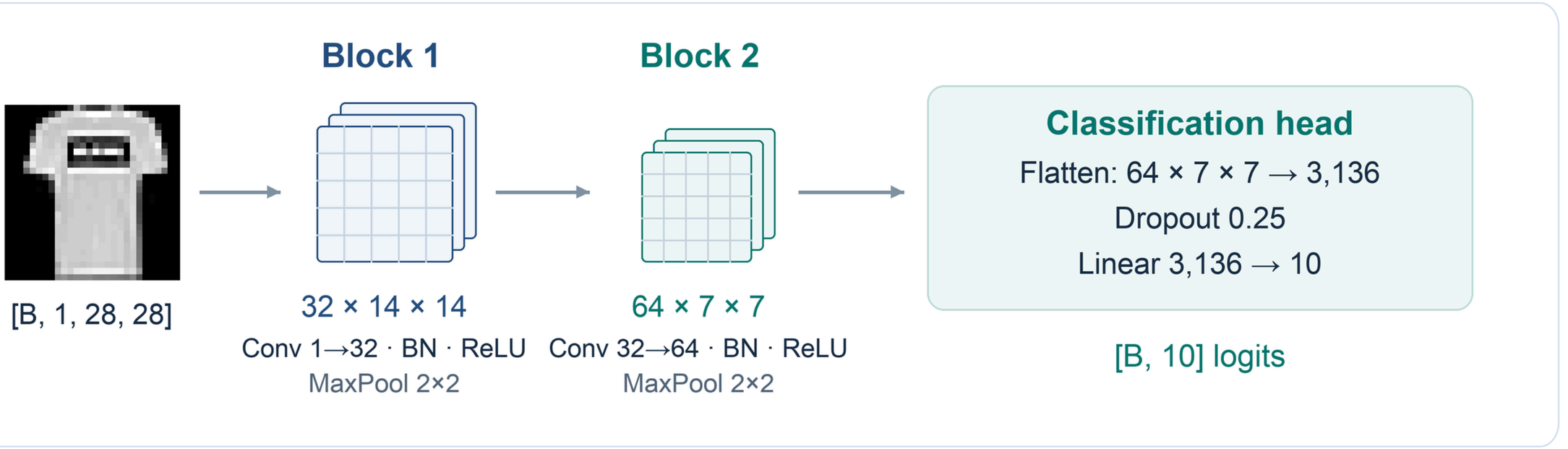
Flattened pixels → 256 → 128 → 10 | Selected neurons shown; dropout is active only during training

Flatten → 784	FC 256 + ReLU	Dropout 0.2	FC 128 + ReLU	FC 10 logits

02 / METHODOLOGY — ARCHITECTURE

Convolutional neural network — architecture

Two spatial feature extraction blocks and a classification head



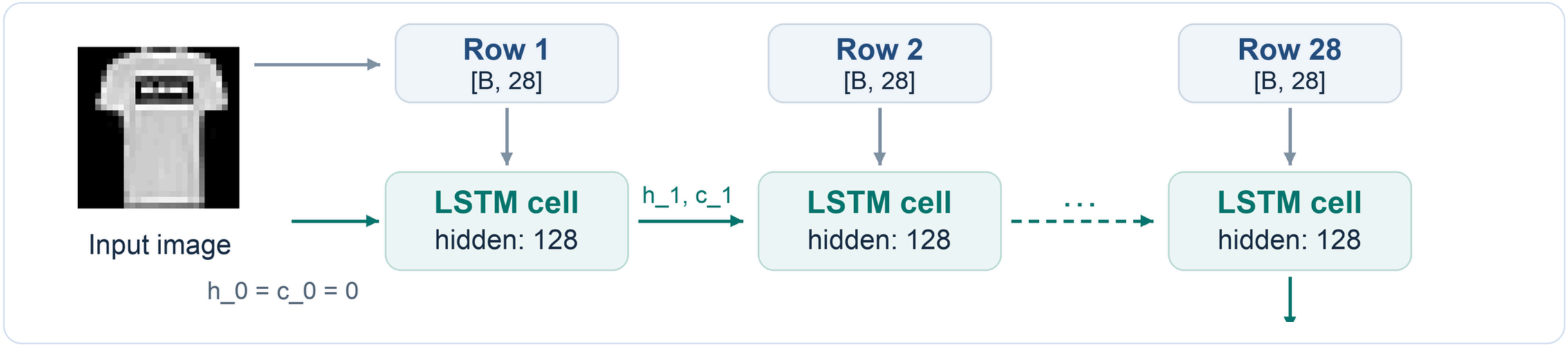
Input 1×28×28	Conv32 + BN + pool	Conv64 + BN + pool	Flatten 3,136	Dropout + FC 10

50,378 parameters · Each 3×3 convolution is followed by BatchNorm, ReLU and 2×2 max pooling.

02 / METHODOLOGY — ARCHITECTURE

LSTM — row-sequence architecture

An image becomes 28 ordered row vectors, followed by recurrent aggregation



28×28 image	Rows: 28 × 28	LSTM, hidden 128	Final hidden h_t	Linear → 10

82,186 parameters for row and column variants; patch input has 92,938 parameters.

02 / METHODOLOGY — ARCHITECTURE

Transformer — patch-encoder architecture

Patch Transformer encoder

540,170 parameters



7×7 patches

16 patch tokens
[B, 16, 49]

Linear 49 → 128
[B, 16, 128]

Add CLS + positions
Dropout 0.1 · [B, 17, 128]

Encoder block × 4

4-head self-attention → residual + LayerNorm

FFN 128→256→128 (GELU) → residual + LayerNorm

CLS output
LayerNorm (128)
Linear 128 → 10
[B, 10] logits

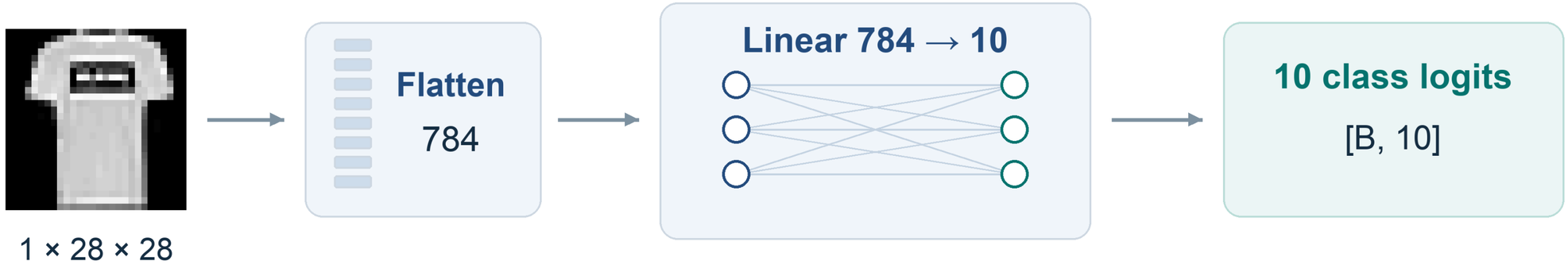
4 heads · d_model = 128 · d_head = 32 · learned positional embeddings · dropout = 0.1

02 / METHODOLOGY

Linear → MLP → CNN: adding inductive bias

Linear / softmax classifier

7,850 parameters



One affine layer · no hidden activation · logits passed directly to cross-entropy

LINEAR

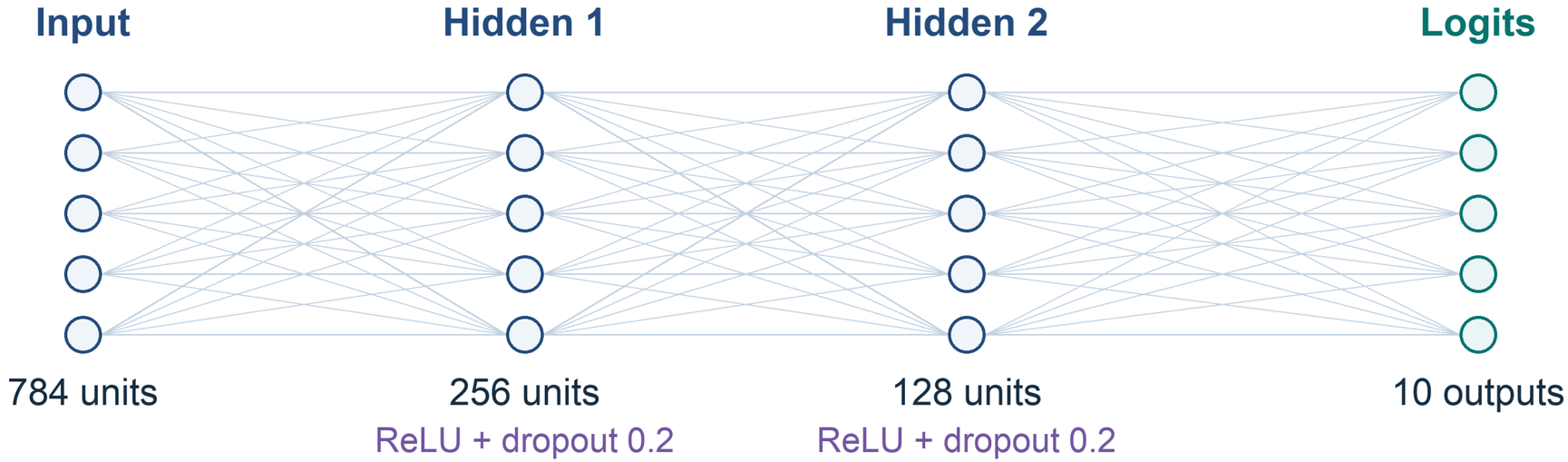
Affine scores of 784 pixels

02 / METHODOLOGY

Linear → MLP → CNN: adding inductive bias

Multilayer perceptron

235,146 parameters



Flattened pixels → 256 → 128 → 10 | Selected neurons shown; dropout is active only during training

MLP

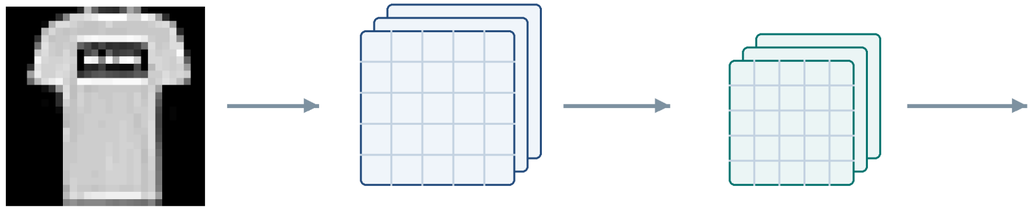
784 → 256 → 128 → 10

02 / METHODOLOGY

Linear → MLP → CNN: adding inductive bias

Convolutional neural network

50,378 parameters



3×3 convolutions, stride 1, padding 1 · pooling halves each spatial dimension · feature maps are schematic

CNN

3×3 conv blocks + pooling

02 / METHODOLOGY

LSTM: reading images as sequences

ROWS

28 steps × 28 features

82,186 params

COLUMNS

28 steps × 28 features

82,186 params

7×7 PATCHES

16 steps × 49 features

92,938 params

One unidirectional LSTM · hidden size 128 · final hidden state → 10-class head

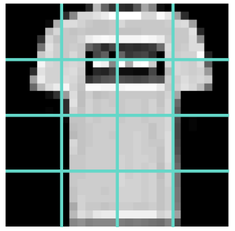
Reshaping preserves all 784 pixels, but changes scan order, recurrent steps and (for patches) model size.

02 / METHODOLOGY

Patch Transformer architecture

Patch Transformer encoder

540,170 parameters



7×7 patches

16 patch tokens
[B, 16, 49]

Linear 49 → 128
[B, 16, 128]

Add CLS + positions
Dropout 0.1 · [B, 17, 128]

Encoder block × 4

4-head self-attention → residual + LayerNorm

FFN 128→256→128 (GELU) → residual + LayerNorm

CLS output
LayerNorm (128)
Linear 128 → 10
[B, 10] logits

4 heads · d_model = 128 · d_head = 32 · learned positional embeddings · dropout = 0.1

02 / METHODOLOGY

Patch Transformer architecture

- 16 non-overlapping 7×7 patches, each projected to dimension 128
- Add learnable CLS token and positional embeddings: 17 tokens
- 4 encoder blocks, 4 attention heads, FFN 128 → 256 → 128

03 / EXPERIMENTS

Experimental setup

Objective	Cross-entropy
Optimizer	Adam, learning rate 0.001
Batch / seed	128 / 42
Budget	Up to 50 epochs
Early stopping	Patience = 5
Checkpoint	Best validation macro-F1
Compute	RTX 4060 Laptop GPU, CUDA

No augmentation, scheduler, mixed precision or pretrained weights in recorded main experiments.

03 / EXPERIMENTS

How evaluation is measured

Accuracy	Fraction of correctly predicted test images
Macro-F1	Mean per-class F1 across 10 classes
Parameters	Trainable model weights and biases
Training time	Fitting, validation and checkpoints
Inference time	Median of 5 full forward-only test passes

Timing uses batch 128 and excludes preprocessing and host-to-device transfer.

04 / RESULTS

Main test results: seven configurations

Model	Acc. (%)	Macro-F1 (%)	Params	Train (s)	Infer. (ms/img)
Linear	84.03	84.08	7,850	508.2	0.00052
MLP	89.38	89.38	235,146	784.8	0.00236
CNN	**91.82**	**91.76**	**50,378**	**367.5**	**0.00349**
LSTM rows	89.92	89.89	82,186	756.6	0.00458
Transformer	88.46	88.44	540,170	502.3	0.01334
LSTM columns	88.61	88.66	82,186	730.2	0.00472
LSTM patches	88.33	88.38	92,938	728.1	0.00374

Source: report Table 6 · Seed 42 · Held-out test set = 10,000 images.

04 / RESULTS

CNN leads predictive performance

91.82%
Test accuracy

91.76%
Test macro-F1

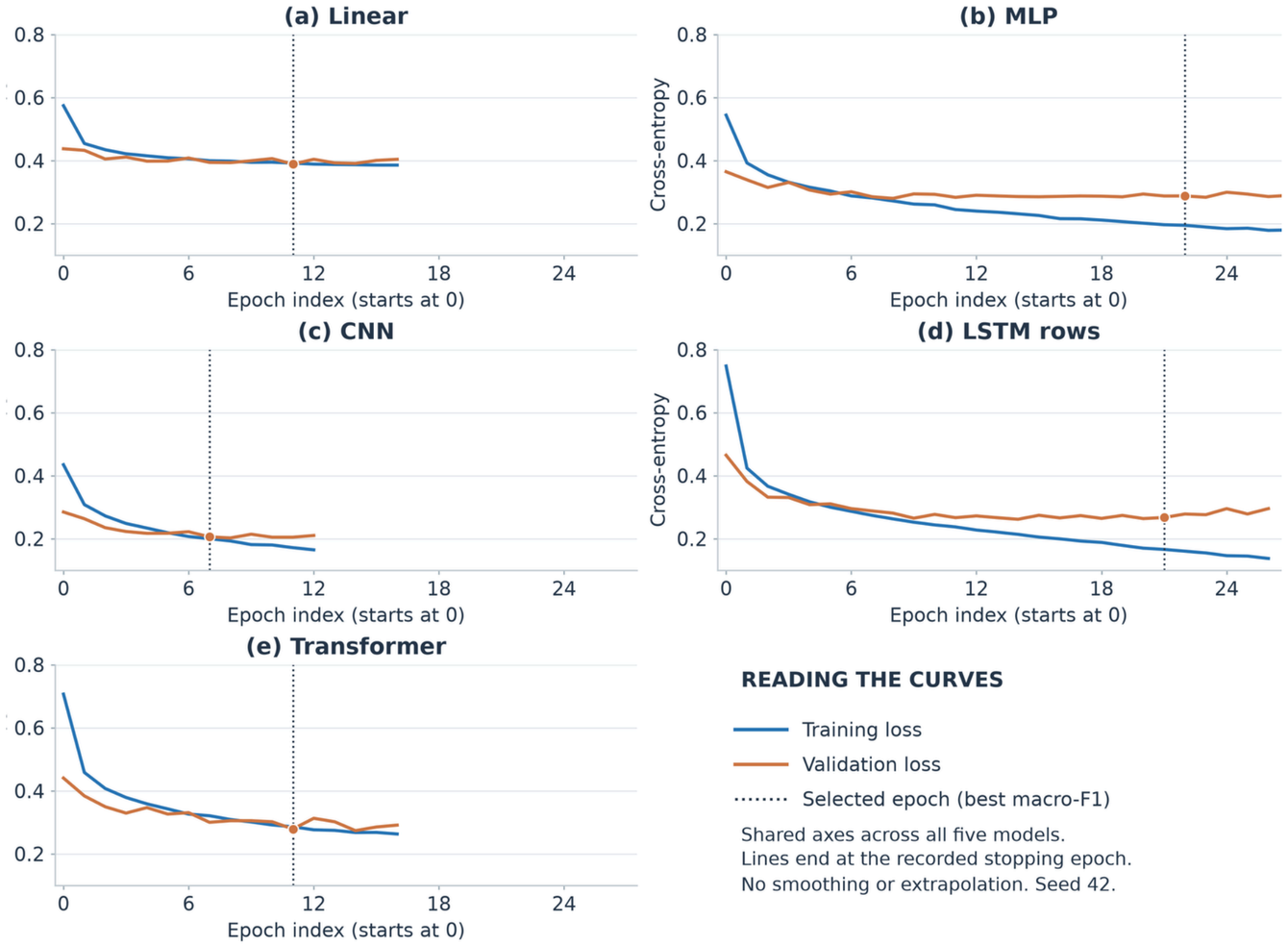
818
Errors out of 10,000

Accuracy by main architecture



04 / RESULTS

Learning curves & checkpoint selection



04 / RESULTS

Learning curves & checkpoint selection

Important observations

- CNN peaks at 92.68% validation macro-F1 (epoch index 7)
- CNN stops after 13 completed epochs
- MLP and row LSTM show larger late train/validation loss gaps
- Best macro-F1 epoch need not minimize validation loss

04 / RESULTS

LSTM representation ablation

	Accuracy	Macro-F1	Forward time/image
Rows	**89.92%**	**89.89%**	0.00458 ms
Columns	**88.61%**	**88.66%**	0.00472 ms
Patches	**88.33%**	**88.38%**	0.00374 ms

Patches are 18.4% faster than rows, but lose 1.59 accuracy points; columns and patches have dirty-run traceability limits.

05 / ERROR ANALYSIS

Classwise performance: Shirt remains difficult

75.54%
CNN Shirt F1

71.50%
CNN Shirt recall

98.95%
CNN Trouser F1

CNN: 285 of 1,000 Shirts are misclassified

- Shirt → T-shirt/top: 112 cases
- Shirt → Coat: 82 cases
- Shirt → Pullover: 61 cases
- These three destinations cover 255 / 285 missed Shirts

05 / ERROR ANALYSIS

Error taxonomy & representative failures

678 82.89%

Garment ↔ garment

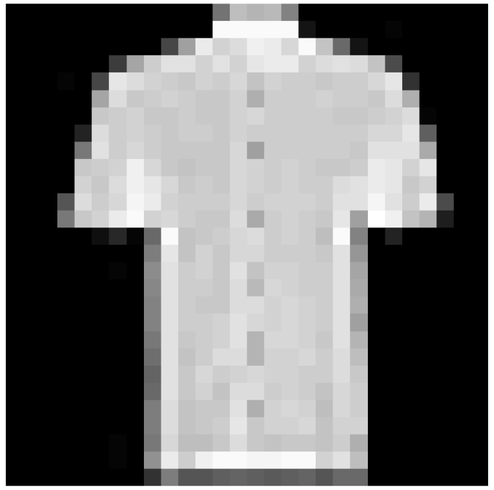
79 9.66%

Footwear ↔ footwear

61 7.46%

Other confusions

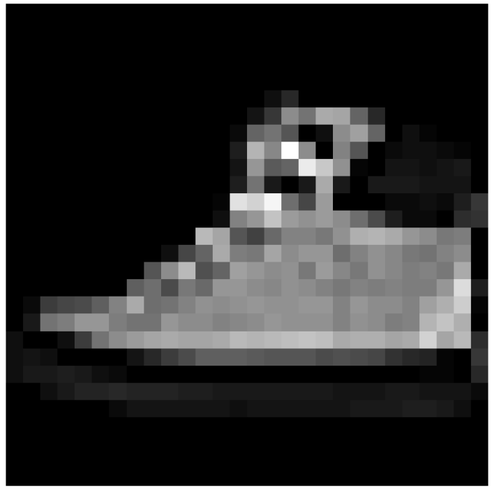
Within G: test #40
CNN



True: Shirt

Pred: T-shirt/top

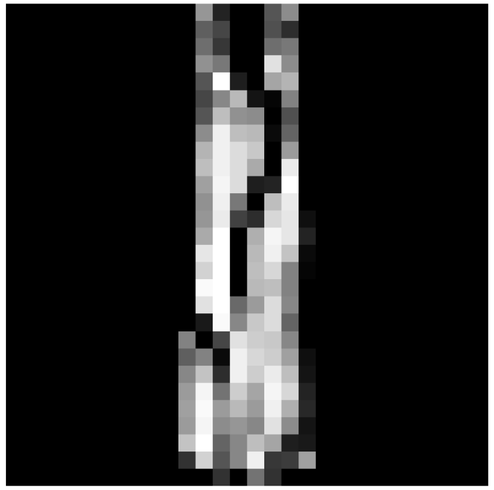
Within F: test #68
CNN



True: Ankle boot

Pred: Sneaker

Other: test #32
CNN

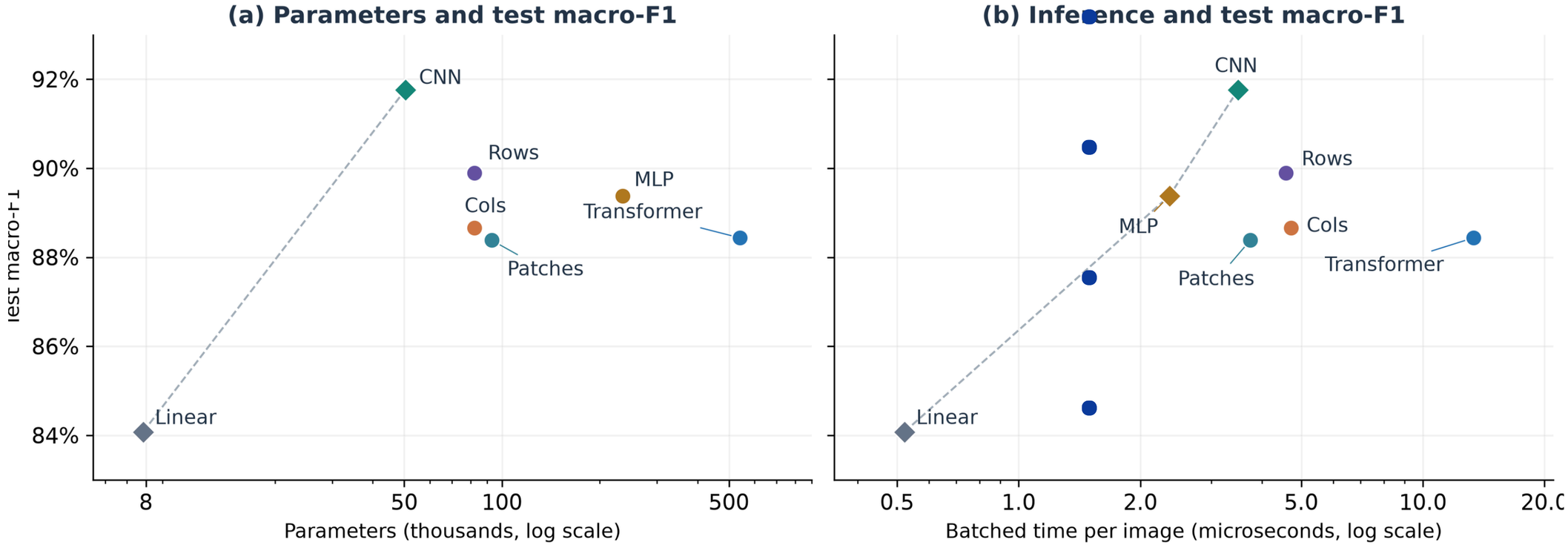


True: Dress

Pred: Trouser

04 / DISCUSSION

Accuracy–efficiency trade-offs



These are observed single-seed settings, not a universal architecture ranking.

04 / DISCUSSION

Accuracy–efficiency trade-offs

- CNN: highest test macro-F1 with only 50,378 parameters
- Linear: smallest parameter count and fastest batched forward time
- MLP: intermediate latency / quality choice
- Transformer: largest model and slowest forward time without best accuracy

These are observed single-seed settings, not a universal architecture ranking.

06 / CONCLUSION

Limitations of the comparison

- Single training seed (42): no variance or significance estimates
- Unmatched parameter budgets, dropout and normalization settings
- No architecture-specific tuning, augmentation or pretrained weights
- Two LSTM variants have incomplete dirty-run source traceability
- Original checkpoints absent; reported inference time is batched and forward-only
- Fashion-MNIST does not establish real-world catalog generalization

06 / CONCLUSION

Conclusion & future work

CNN is the best recorded aggregate-quality choice

91.82% accuracy · 91.76% macro-F1 · 50,378 parameters

Next steps

- Repeat with multiple seeds and matched model capacities
- Improve difficult garment classes with controlled augmentation/resolution tests
- Tune Transformer and sequence models using validation-only selection
- Preserve complete checkpoints, logits and per-image predictions

APPENDIX / OPTIONAL DEMO

Reproducibility & project resources

Implementation

PyTorch training/evaluation pipeline; saved split and configuration snapshots; per-run histories, confusion matrices, and recorded metrics.

Repository

github.com/nguyenhohongphuc/co3133-deep-learning-project

Optional live demo: navigate source code, archived metrics and example plots. Do not claim missing original checkpoints can be loaded.

THANK YOU

Questions & discussion